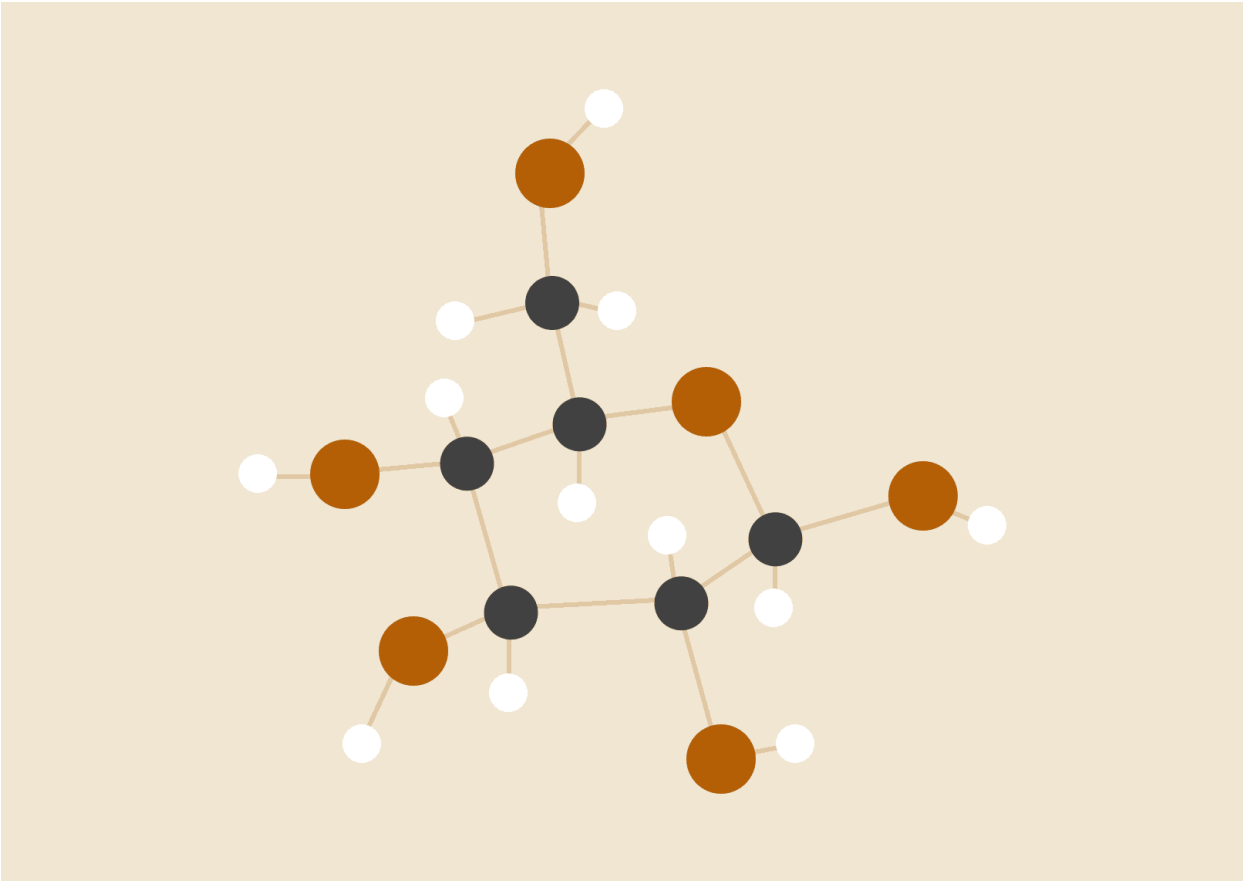
DSS-II Project Report



Enzo Ma

08.04.2026

INTRODUCTION

This project involves the development of a Todo management API as part of the DSS2 course. The application provides endpoints for user authentication, allowing users to securely register and log in, as well as manage their personal todo items.

The API supports both public and private access to tasks. Public todos can be viewed without authentication, while authenticated users have capabilities (such as Create, Read, Update, Delete) over their own private todos. The system ensures proper data handling, validation, and security through the use of modern backend practices such as JWT-based authentication and structured service architecture.

Additionally, the project is configured to run in a standardized development environment, complying with the course requirements, including operating on the specified port 3087 (for cypress testing).

The goal of this report is to deliver the missing pieces of the project along with small context provided by myself. In this report, you will go through the instructions of how to run the project, the required screenshots for Swagger along with the ones for the Database (accompanied by comments).

HOW TO RUN THE PROJECT

How to run Docker:

1. Install Docker Desktop
2. Open a terminal at the project root (DSS-II-Project)
3. Run: docker compose up --build
4. Frontend: http://localhost:3000
5. API: http://localhost:3087

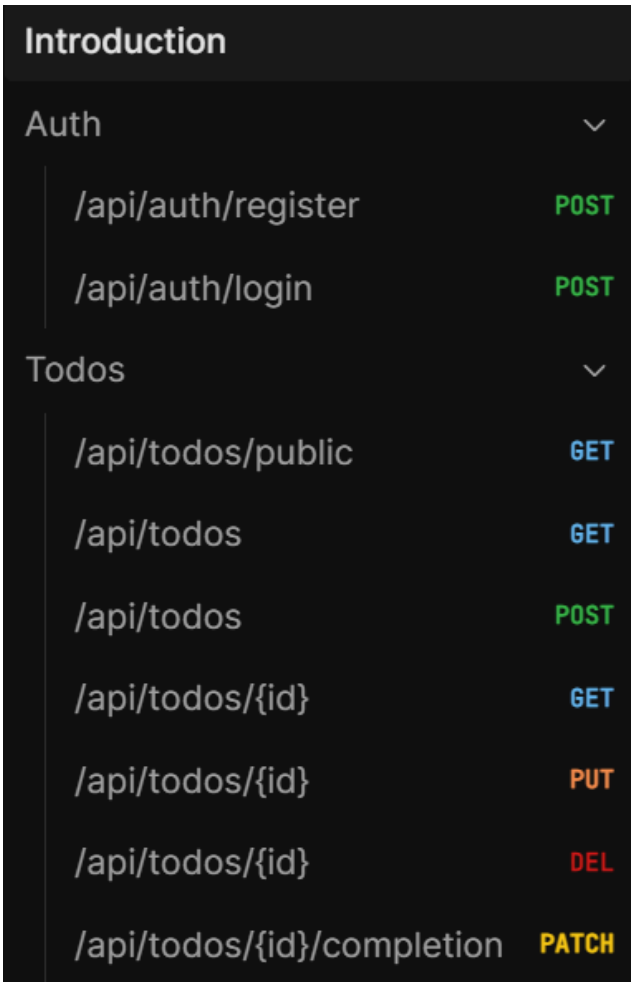
How to run locally:

1. Install .NET 10 SDK
2. Install PostgreSQL 16
3. Create a database named "tododb" in pgAdmin
4. Navigate to dss2-backend/TodoApi/
5. Run: dotnet run
6. API is available at http://localhost:3087

As for DB setups and Migrations: The database schema is managed using Entity Framework Core migrations, which keep it in sync with the application models. In local development, migrations are applied manually using the dotnet ef database update command. When running the application with Docker, migrations are executed automatically on startup, ensuring the database is always up to date.

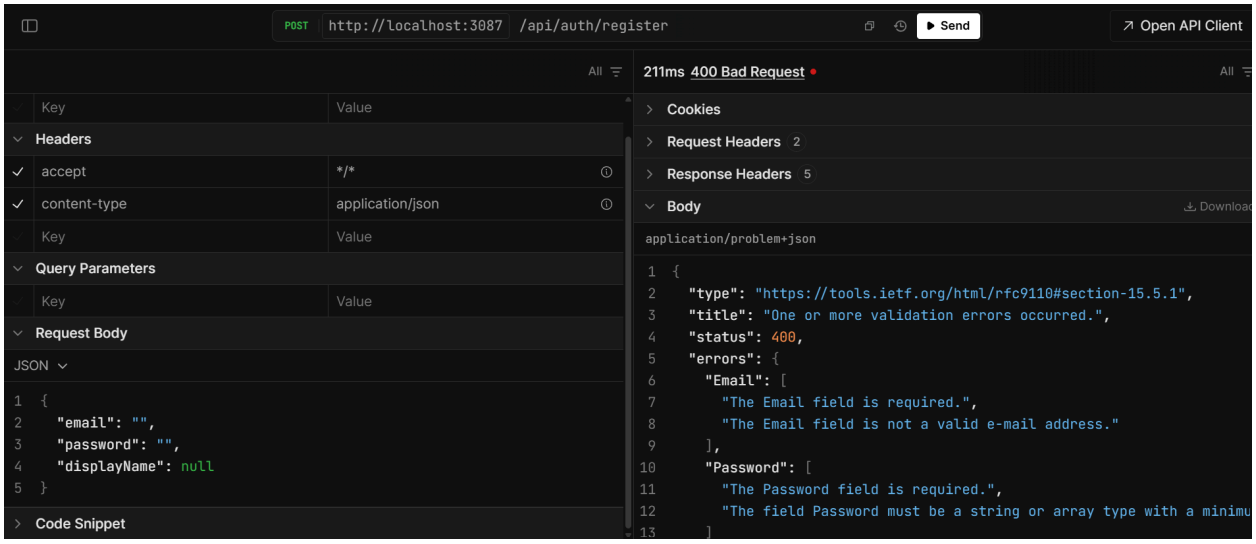
REQUESTS & RESPONSES AND SWAGGER SCREENSHOTS (WITH API ENDPOINTS)

First of all, here below is the Swagger page list, with the endpoints and HTTP actions:

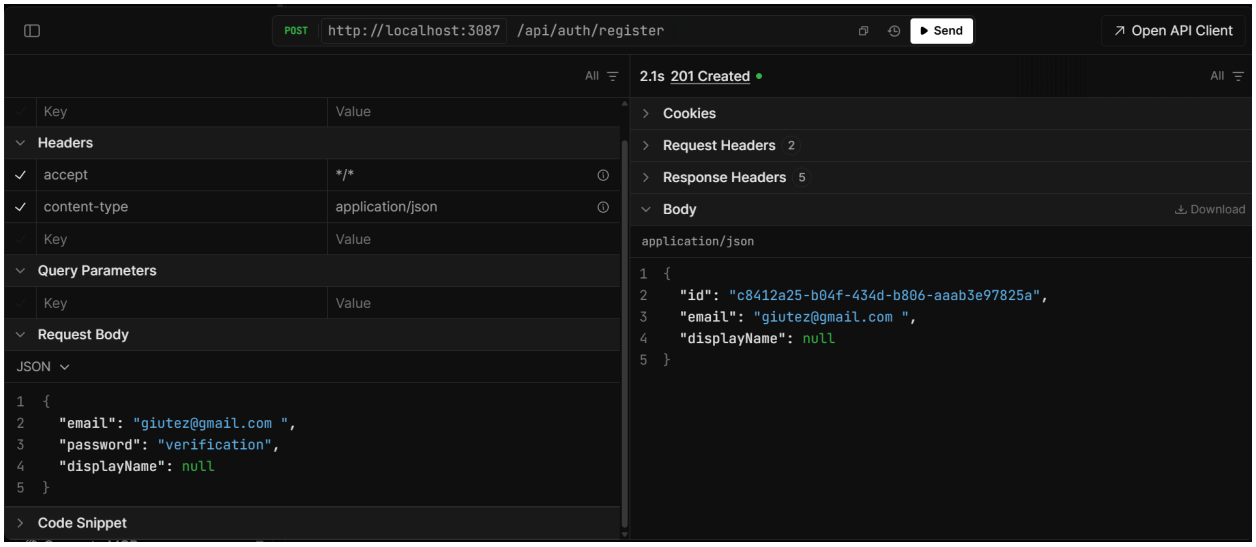


User Authentication:

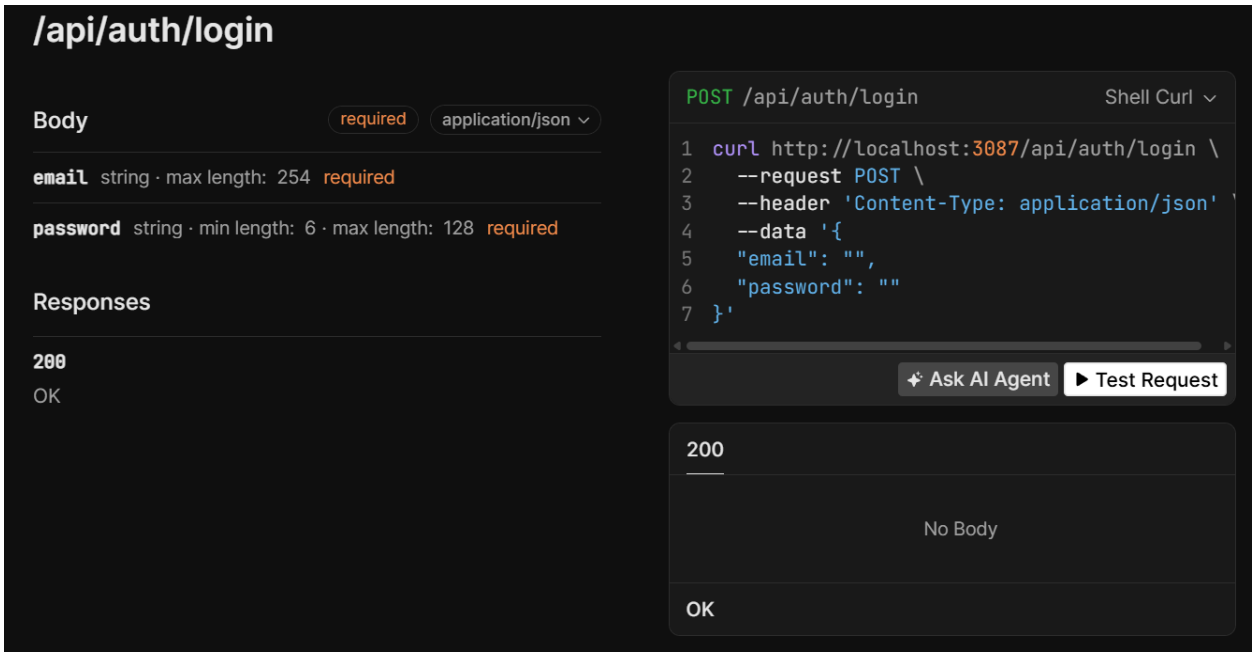
For the registration authentication, we can see here below the return “400 - Bad Request” (a client server error) as the two key elements haven’t been completed (email and password).



However, if both elements are given, it ends up being successful:

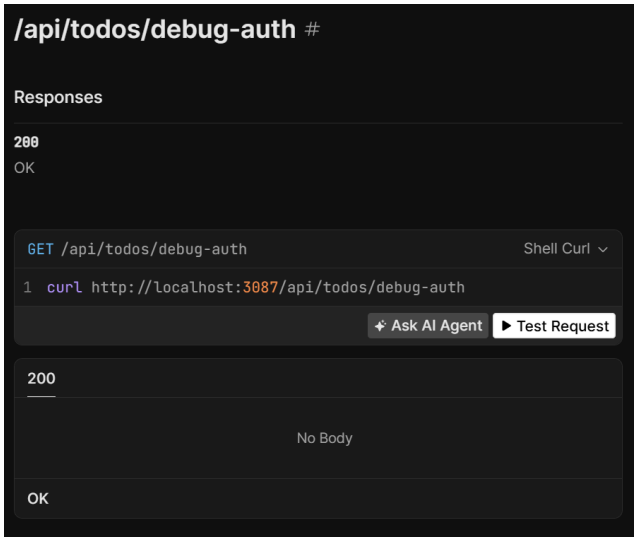


The login works according to different criterias but when they are respected, it ends up working (200 - Ok which translates to everything work).

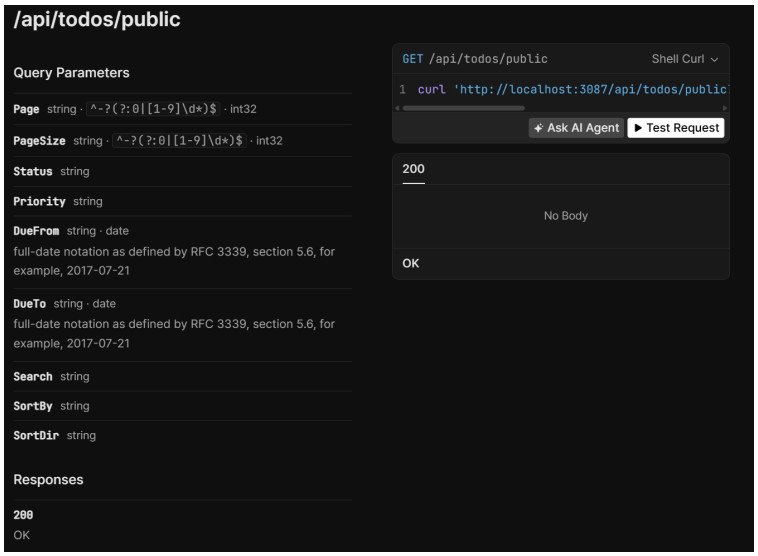


Todo Validations:

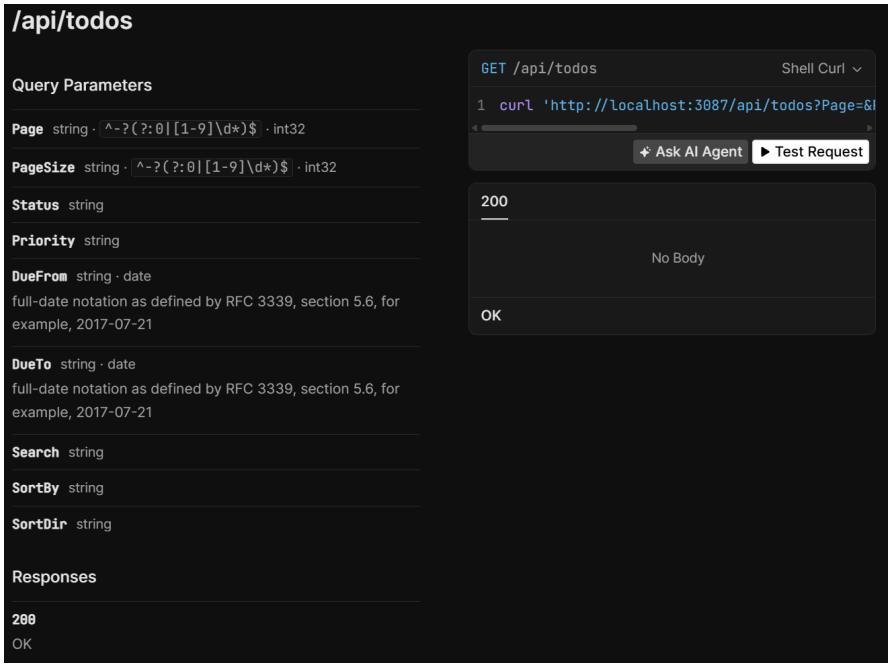
Here below is the proof that all todo actions function correctly.



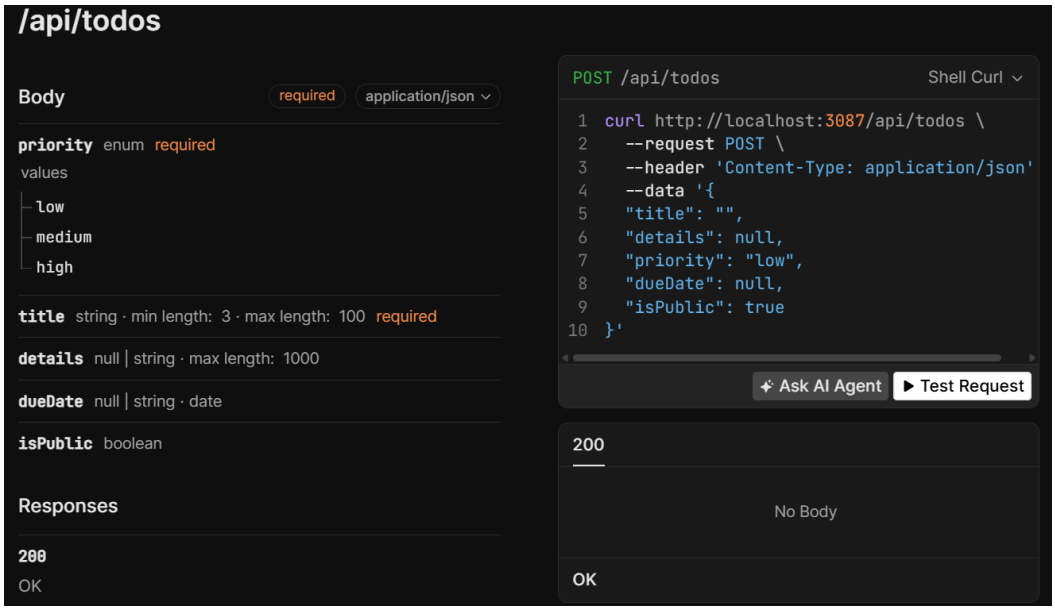
We have the possibility to find a public todo (no matter if logged in or not)



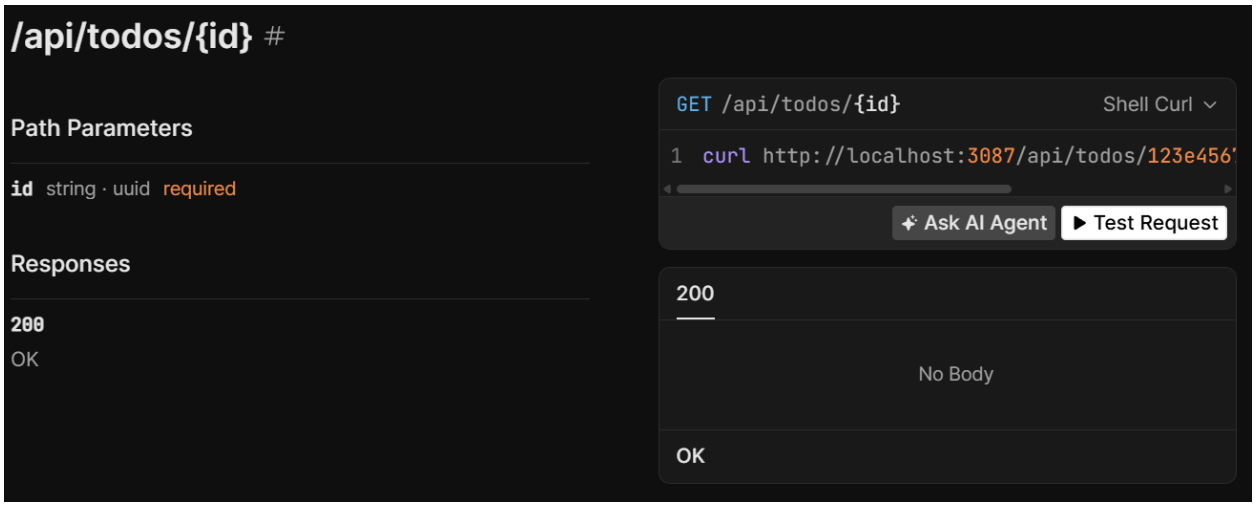
Then we have the same demand (to get a todo Item) but this time a specific one belonging to a specific user logged-in.



The next one is for the publication of a todo (POST).



This one is for finding a specific todo through its id.



This one is for the modification of a todo item.

/api/todos/{id}

Path Parameters

id string · uuid required

Body

required application/json

priority enum required

values

- low
- medium
- high

title string · min length: 3 · max length: 100 required

details null | string · max length: 1000

dueDate null | string · date

isCompleted boolean

isPublic boolean

Responses

200

PUT /api/todos/{id} Shell Curl

```
curl http://localhost:3087/api/todos/123e4
  --request PUT \
  --header 'Content-Type: application/json
  --data '{
  "title": "",
  "details": null,
  "priority": "low",
  "dueDate": null,
  "isPublic": true,
  "isCompleted": true
}'
```

Ask AI Agent ▶ Test Request

200

No Body

OK

Whereas this one is for the removal of a todo item.

/api/todos/{id}

Path Parameters

id string · uuid required

Responses

200

OK

DELETE /api/todos/{id} Shell Curl

```
curl http://localhost:3087/api/todos/123e456
  --request DELETE
```

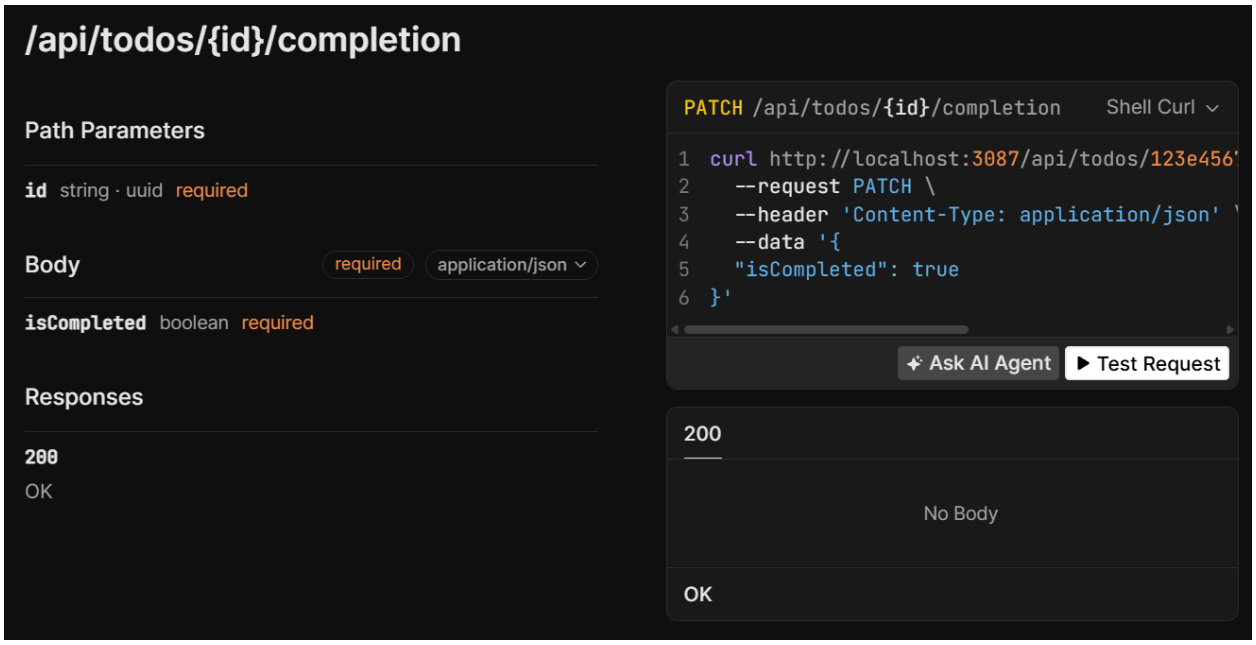
Ask AI Agent ▶ Test Request

200

No Body

OK

Finally, this one is for once the todo item is completed.



DATABASE (WITH SCREENSHOTS)

For the database, I used pgAdmin4 with POSTGRE16 as the server setup (along with some additional changes made through C#).

We can find two tables in the database:

- User Table: This one is the table that stocks user information such as UserID (primary key), email, password (hashed), display name and a timestamp for the creation of the account.
- Todo Items Table: It contains ID(as a primary key), UserID(as a foreign key), Title, Details, Priority, due date, completion status, public status and two timestamps (for creation and latest update).

Here below are the screenshots of a few rows of each table. We can see the column names along with their types (and which is a primary key, foreign key, or neither).

Users Table:

	Id [PK] uuid	Email character varying (254)	PasswordHash text	DisplayName character varying (100)	CreatedAt timestamp with time zone
1	00d90c31-cf2a-40b2-87ea-14aff11d7b...	pub_1775048378850@example.co...	$2a$12$ppVye0nIZWnAx6UQ2hd.jeX1LYIDAN6xVlCVmC5vW0wlWv...		2026-04-01 15:59:41.758847+...
2	01703558-c17e-42e5-8b0d-4d9494dc4...	cytest_1774981236_0@example.c...	$2a$12$EVaMjFpMiKuTxSXKvl6t6Oz.hz3KIVmKiCF7mKoFiVXFQUPL...	cypress	2026-03-31 21:20:36.909764+...
3	02dfc1bd-618b-448d-8cdb-6460d5e06...	pub_1774992520359@example.co...	$2a$12$RZPhqXozXclWTRNN7uDqi.BnOgRQ7Fm.0ZkwWNoDPoe/3...		2026-04-01 00:28:40.397426+...
4	0388ccb6-e02d-43e7-9596-8ba1e9b39...	userA_1774983139766@example....	$2a$12$D1hQ1bQmzhFUIJDW5c1KVuUJJFOJi8EmklnlbsD7r3fhDxG...		2026-03-31 21:52:21.157129+...
5	0583bec7-ba7b-4fa5-8d44-78fce3e8e8...	userA_1775046342670@example....	$2a$12$AfDAA2j9mFkAT1mDp3GeE.9OPPjp4PpDRh33.dsRCy9xck/...	Jeff	2026-04-01 15:25:46.362497+...
6	063b9207-140e-4a9a-a304-f08500d8a...	mynameJeff@gmail.com	$2a$12$oZvg0JyXTFn7u1rNymtD4.1ZzIMkXY5GbVxd6jujojUsE0u4C...		2026-04-02 15:01:50.583721+...
7	06da9de1-67c6-4cbd-ae94-be6c23c86...	iso1_1774983094464@example.c...	$2a$12$w4XegZl.om0/zUzYXBnKjuzQ7OyZdOfFHlm9T0V.zPkYzQKC...		2026-03-31 21:51:34.477094+...
					2026-04-01 15:42:20.674677+...
					2026-04-01 15:59:48.047066+...
					2026-03-30 11:17:17.803916+...

Todo Items Table:

	Id [PK] uuid	UserId uuid	Title character varying (100)	Details character varying (1000)	Priority text
1	03aa3677-0541-4266-8ca1-07a12c87e...	2c40c96b-cfab-467e-adb4-dbb285fb72...	Paged 6		low
2	041e4a8f-4b48-450b-a399-99914174e...	63792068-ecd5-4e6d-b42f-b90ae6db1...	Paged 11		low
3	06066396-499f-47f3-9d55-ee997a113...	8bff217a-45c4-4416-8a6b-94ad527ce...	Paged 20		low
4	0782c552-6ae5-44c5-ad46-c3cea9894...	79fa4bc9-7ff2-4940-95bd-ce35530441...	Public todo for guests	visible	low
5	08cbdf53-3b97-4810-846c-8f4ccd960...	2c40c96b-cfab-467e-adb4-dbb285fb72...	Paged 11		low
6	0a731359-d600-425f-8be3-dc175b66d...	8bff217a-45c4-4416-8a6b-94ad527ce...	Paged 6		low
7	0a97fb07-30af-4473-98a5-2dbfbf5bd8...	dd1bc514-6b56-4ca0-9ff3-9882a313fc...	Low priority item		low
8	0e0d95f5-a719-4ea6-a1d0-bbd6d2d45...	074698f1-a6d3-4d74-9907-f5974c967...	Private todo	should be private	medium
9	0e218c40-8988-430d-bc94-2d4518a2...	2c40c96b-cfab-467e-adb4-dbb285fb72...	Paged 22		low
10	1087cb62-e7c2-4499-8a53-51ad4861...	5e1e223d-8c89-4d48-80c4-e61eb0d1c...	High priority item		high
11	11fbccca-0d58-4442-9902-971a4abc8...	2c40c96b-cfab-467e-adb4-dbb285fb72...	Paged 21		low
12	1462dfcf-65cb-4d53-992a-55be905f12...	2c40c96b-cfab-467e-adb4-dbb285fb72...	Paged 10		low
13	149f1ece-6031-418e-a044-7dfa8b681...	4064384f-9b5d-4d69-8c6b-a3afef667e...	Beta	something else	low
14	17cb6e09-2d00-40d2-9ac2-fef9251d8f...	63792068-ecd5-4e6d-b42f-b90ae6db1...	Paged 14		low
15	1848ee15-45a8-4971-8ab6-1a86a408e...	2c40c96b-cfab-467e-adb4-dbb285fb72...	Paged 0		low

DueDate date	IsCompleted boolean	IsPublic boolean	CreatedAt timestamp with time zone	UpdatedAt timestamp with time zone
[null]	false	false	2026-04-01 16:00:19.85913+03	2026-04-01 16:00:19.859131+...
[null]	false	false	2026-04-01 18:21:23.683364+...	2026-04-01 18:21:23.683364+...
[null]	false	false	2026-04-05 22:31:19.676367+...	2026-04-05 22:31:19.676367+...
[null]	false	true	2026-04-01 15:59:54.088246+...	2026-04-01 15:59:54.088246+...
[null]	false	false	2026-04-01 16:00:19.961182+...	2026-04-01 16:00:19.961182+...
[null]	false	false	2026-04-05 22:31:19.573655+...	2026-04-05 22:31:19.573655+...
[null]	false	false	2026-04-01 16:00:32.67648+03	2026-04-01 16:00:32.676481+...
[null]	false	false	2026-04-01 15:59:49.162948+...	2026-04-01 15:59:49.162949+...
[null]	false	false	2026-04-01 16:00:20.184067+...	2026-04-01 16:00:20.184067+...
[null]	true	false	2026-04-01 18:38:43.023194+...	2026-04-01 18:38:43.054083+...
[null]	false	false	2026-04-01 16:00:20.163564+...	2026-04-01 16:00:20.163565+...
[null]	false	false	2026-04-01 16:00:19.941246+...	2026-04-01 16:00:19.941246+...
[null]	false	false	2026-04-05 22:32:14.309914+...	2026-04-05 22:32:14.309914+...
[null]	false	false	2026-04-01 18:21:23.718614+...	2026-04-01 18:21:23.718615+...
[null]	false	false	2026-04-01 16:00:19.694084+...	2026-04-01 16:00:19.694084+...

THE END